

Flight Booking System

Introduction:

The study provides a complete description of the design and implementation of a SQL Server-based database system for airline ticketing and reservations. The system's major goal is to support the entire flight booking, ticket issuing, luggage handling, staff authentication, and service upgrades process from start to finish. The implementation adheres to best standards for relational database design, data normalisation, query optimisation, transaction management, and security enforcement.

The database was created with T-SQL and has a high degree of normalisation (up to 3NF), a modular design with stored procedures, views, and functions, and security features such as access control and data validation. Real-world business use cases, such as recognising high-value passengers, enforcing baggage limits, managing employee responsibilities, and accessing reservation data dynamically, are implemented using queries and database objects. Transactions, triggers, and restrictions have been used to handle issues such as concurrency, data integrity, and error handling.

The report describes the database schema and implementation in depth, presents solutions to specified query jobs (Q2-Q8), and examines the practical consequences of database integrity, performance, and security in the context of a real-world airline.

Part 1: Database Design and Normalization

Understanding the Requirements

The client, an airport authority, is in the process of establishing a secure and efficient electronic ticketing system to facilitate the issuance of e-boarding numbers to passengers with pre-booked flights. The system must collect detailed flight-related data and support operational functions required for current airline operations. Initial consultations have resulted in the following functional and technical requirements:

- **User Authentication and Role-Based Access**
 - Employees must authenticate themselves at the start of each shift using unique login credentials. The system must provide secure access via login validation. In the event of failed authentication, the Ticketing Supervisor has the authority to override the process and give system access, assuring operational continuity.
- **Passenger Reservation and Ticket Management**
 - The system must handle passenger bookings using a unique identification known as the Passenger Name Record (PNR), which contains the traveller's itinerary and reservation status (confirmed,

pending, or cancelled). Each reservation is associated with a specific aircraft and contains information such as departure and arrival schedules, seat preferences, and booking data. Passengers might have several reservations.

- Ticket Issuance and E-Boarding Number Generation
 - After confirming a reservation, the employee is responsible for issuing a ticket and producing an e-boarding number. This ticket is closely related with the passenger's PNR and the employee who issued it. If a preferred seat is unavailable, the seat number field should be marked as NULL. The procedure must assure correct data entry and association of all linked businesses.
- Additional Services and Fare Computation
 - The system must accommodate optional services requested by passengers, which influence the total fare. These services include the following:
 - Extra luggage is £100 per kg beyond the 20kg free allowance.
 - Upgraded lunch costs £20 per person.
 - Preferred seating is offered at £30 per person.
 - Additional services should be listed and linked to individual tickets.
- Flight Details and Baggage Handling
 - A single flight may be associated with several tickets and reservations. The baggage handling system must keep track of each item's state (e.g., checked-in or loaded), weight, and any associated baggage taxes. Baggage data must be directly linked to the corresponding ticket and flight to ensure accurate traceability and operational efficiency.
- Database System Capabilities
 - The new database system must meet all of the above operational requirements by providing the following important capabilities:
 - Data integrity and referential consistency are guaranteed by primary and foreign key relationships.
 - Advanced database objects like stored procedures, views, user-defined functions, and triggers are used to contain business logic and automate mundane tasks.
 - Security measures to secure sensitive data, including login credentials and personal passenger information.
 - Support multiple users without affecting data accuracy or performance.
 - Scalable architecture enables future enhancements and the addition of new services, such as real-time flight monitoring and payment gateway integration.

Normalization Process

The database design approach followed normalisation principles till the Third Normal Form (3NF). This ensures that data redundancy is reduced while integrity is preserved. The normalisation steps included:

- The First Normal Form (1NF) ensures atomicity by storing only one value per field. Employee names and emails, for example, are organised into separate columns.
- The Second Normal Form (2NF) eliminated partial dependencies by segregating authentication data from the core employee table.
- Third Normal Form (3NF): Removed transitive dependencies; for example, passwords and salts are only dependent on the authentication table's main key, not any non-key properties.

Primary Database Entities

The schema has the following tables:

- Passenger: Saves personal information and preferences for passengers.
- Flight: Contains information about flight schedules, origins, and destinations.
- Reservation: connects passengers to specific flights and includes booking information.
- Ticket: A finalised reservation that includes class, fare, and seat number.
- Employee: Stores employee information used in ticket issuance.
- Baggage: Monitors luggage data and status.
- Additional Services: Includes optional services including upgraded meals, extra baggage, and seat preferences.
- Employee Authentication: Handles login credentials for system access.

The relationships are defined by primary and foreign keys, which ensure data consistency across transactions and activities.

Database Schema Explanation

The database architecture for the airport e-boarding and ticketing system consists of eight interconnected tables, each of which captures an important aspect of the operational workflow. The architecture uses a relational paradigm to ensure data normalisation (3NF), integrity, and scalability. Below is an explanation of each table's job and how they relate to one another:

1. Passenger

- Purpose: Stores personal and travel-related information about each passenger.
- Attributes:
 - PassengerId (Primary Key)
 - PNR (unique booking reference)
 - FirstName, LastName, Email
 - MealPreference (Vegetarian/Non-Vegetarian)
 - EmergencyContact (optional)

- DOB (Date of Birth)
- Relationships:
 - One Passenger can have many Reservations.
 - Linked to Reservation via PassengerId.

2. Flight

- Purpose: Represents details about each flight.
- Attributes:
 - FlightId (Primary Key)
 - FlightNumber, DepartureTime, ArrivalTime
 - Origin, Destination
- Relationships:
 - One Flight can have many Reservations.
 - Linked to Reservation via FlightId.

3. Reservation

- Purpose: Captures the booking action of a passenger for a particular flight.
- Attributes:
 - ReservationId (Primary Key)
 - PassengerId (FK from Passenger)
 - FlightId (FK from Flight)
 - ReservationDate
 - BookingStatus (Confirmed, Pending, Cancelled)
- Relationships:
 - One Reservation can result in one Ticket.
 - Connects Passenger and Flight.

4. Ticket

- Purpose: Finalized issuance of a seat on a flight for a confirmed reservation.
- Attributes:
 - TicketId (Primary Key)
 - ReservationId (FK from Reservation)

- EmployeeId (FK from Employee)
- IssueDate, IssueTime
- Fare, SeatNumber, Class (Economy, Business, First Class)
- Relationships:
 - One Ticket is linked to one Reservation.
 - Issued by an Employee.
 - Can have multiple Baggage records.
 - Can have a single AdditionalServices record.

5. Employee

- Purpose: Stores data of employees who manage ticketing operations.
- Attributes:
 - EmployeeId (Primary Key)
 - EmployeeName, Email
 - Role (Ticketing Staff, Ticketing Supervisor)
- Relationships:
 - One Employee may issue many Tickets.
 - Linked to Ticket and EmployeeAuthentication.

6. EmployeeAuthentication

- Purpose: Handles login credentials for employees.
- Attributes:
 - AuthId (Primary Key)
 - EmployeeId (FK from Employee)
 - Username, Password
- Relationships:
 - One-to-one with Employee.

7. AdditionalServices

- Purpose: Tracks optional services linked to a ticket.
- Attributes:
 - ServiceId (Primary Key)

- TicketId (FK from Ticket)
- ExtraBaggage (bit)
- UpgradedMeal (bit)
- PreferredSeat (bit)
- TotalAdditionalFare
- Relationships:
 - Each Ticket may have one record in AdditionalServices.

8. Baggage

- Purpose: Manages passenger baggage details.
- Attributes:
 - BaggageId (Primary Key)
 - TicketId (FK from Ticket)
 - BaggageWeight, BaggageStatus, BaggageFee
- Relationships:
 - Multiple Baggage entries can be linked to one Ticket.

Summary of Key Relationships

- One Passenger → Many Reservations
- One Reservation → One Ticket
- One Flight → Many Reservations
- One Employee → Many Tickets
- One Ticket → One AdditionalServices
- One Ticket → Many Baggage entries

```

1 | -- Creating Database
2 | Create Database FlightBookingSystem;
3 |
4 | -- Using FlightBookingSystem database
5 | use FlightBookingSystem;
6 |

```

Table Creation and Explanation

The database was built using a number of normalised tables, each with careful consideration for data types, constraints, and relational integrity. Each table represents a specific entity or function required by the airport's ticketing and reservation system. The following is a summary of the primary tables developed, along with an explanation of their structure and purpose.

Passenger Table

This table contains each passenger's personal and travel preference data. Fields contain identifiers such as PassengerID and PNR, as well as necessary information such as first and surname name, email, date of birth, food preference, and an optional emergency contact number.

- The primary key, PassengerID, assures uniqueness.
- PNR (Passenger Name Record) is a unique identifier used for bookings.
- Meal preferences are constrained to specified values (vegetarian/non-vegetarian) via a tick restriction.
- Nullable fields, such as the emergency contact number, offer versatility.

```
7 | -- Tables: Passenger, Employee, Flight, Ticket, Reservation, Baggage, Additional Services
8 | -- Passenger Table
9 | Create table Passenger(
10 |     PassengerId int primary key identity(1,1),
11 |     PNR nvarchar(10) unique not null,
12 |     FirstName nvarchar(50) not null,
13 |     LastName nvarchar(50) not null,
14 |     Email nvarchar(50) unique not null,
15 |     DateOfBirth date not null,
16 |     MealPreference nvarchar(15) check (MealPreference in ('Vegetarian', 'Non-Vegetarian')),
17 |     EmergencyContact nvarchar(20) null)
18 |
```

Flight Table

This table contains detailed flight information such as the unique FlightID, flight number, origin and destination airports, and departure and arrival timings. Designed to accept multiple bookings and tickets per flight. Constraints guarantee that values be rational and consistent, such as the arrival time being after departure.

```
19 | -- Flight Table
20 | Create table Flight(
21 |     FlightId int primary key identity(1,1),
22 |     FlightNumber nvarchar(10) unique not null,
23 |     Origin nvarchar(20) not null,
24 |     Destination nvarchar(20) not null,
25 |     DepartureTime Datetime not null,
26 |     ArrivalTime Datetime not null)
27 |
```

Reservation Table

This is the bridge table that connects passengers and flights. It saves the reservation status (pending, confirmed, or cancelled), the reservation date, and assigns the booking to a specific passenger and flight.

- A foreign key constraint connects both the Passenger and Flight tables.
- To guarantee that reservation states are valid, status is restricted using a check constraint.
- The reservation date includes a constraint that prevents backdated entries.

```

28 -- Reservation Table
29 Create table Reservation(
30     ReservationId int primary key identity(1,1),
31     PNR nvarchar(10) not null foreign key(PNR) references Passenger(PNR),
32     PassengerId int not null foreign key(PassengerId) references Passenger(PassengerId),
33     FlightId int not null foreign key(FlightId) references Flight(FlightId),
34     BookingDate Date not null check(BookingDate >= getdate()),
35     BookingStatus nvarchar(10) check(BookingStatus in ('Confirmed', 'Pending', 'Cancelled'))
36

```

Ticket Table

This is the finalised form of a reservation. It includes important information such as the TicketID, class type (economy, business, or first class), seat number, fare, issue date/time, and the issuing employee's ID.

- Foreign key limitations associate the ticket with its reservation and issuing employee.
- The class type is validated using a check constraint.
- The seat number is nullable, which allows for flexibility when a preferred seat is unavailable.

```

37 -- Ticket Table
38 Create table Ticket(
39     TicketId int primary key identity(1,1),
40     ReservationId int not null foreign key(ReservationId) references Reservation(ReservationId),
41     IssueDate Date default getdate(),
42     IssueTime Time default getdate(),
43     SeatNumber nvarchar(5) null,
44     Class nvarchar(15) check(Class in ('Economy', 'Business', 'First-Class')),
45     Fare decimal(10,2) not null,
46     EmployeeId int not null foreign key(EmployeeId) references Employee(EmployeeId))
47

```

Employee Table

Stores information about ticketing employees and supervisors, such as their name, email address, role (staff or supervisor), and a unique employee ID.

- A role field is evaluated using restrictions to ensure suitable access levels.
- The EmployeeID is used in the Ticket table to determine who issued the ticket.

```

48 -- Employee Table
49 Create table Employee (
50     EmployeeId int primary key identity(1,1),
51     EmployeeName nvarchar(100) not null,
52     EmployeeEmail nvarchar(100) unique not null,
53     EmployeeRole nvarchar(20) check (EmployeeRole in ('Ticketing Staff', 'Ticketing Supervisor'))
54 );
55

```

Employee Authentication Table

A security-oriented table that stores usernames and passwords for system access. It connects to the Employee table via EmployeeID. Helps to enforce user authentication and access control. Provides basic role-based permissions and audit features.


```

73 -- Authentication table for storing staff login
74 Create table EmployeeAuthentication (
75     AuthId int primary key identity(1,1),
76     EmployeeId int not null foreign key(EmployeeId) references Employee(EmployeeId), -- Foreign key reference to Employee table
77     Username nvarchar(50) not null, -- Username for login
78     HashedPassword binary(64) not null, -- Hashed password
79     Salt nvarchar(40) not null, -- Salt for password hashing
80 );
81
82

```

Baggage Table

This table keeps track of the baggage assigned to each ticket, including its weight, status (checked in or loaded), and price.

- A foreign key links it to the Ticket.
- A trigger is used to enforce a business requirement, requiring that baggage weighing more than 20kg be accompanied by the appropriate baggage cost.

```

56 -- Baggage Table
57 Create table Baggage (
58     BaggageId int primary key identity(1,1),
59     TicketId int not null foreign key(TicketId) references Ticket(TicketId),
60     BaggageWeight decimal(5,2) check (BaggageWeight >= 0),
61     BaggageStatus nvarchar(10) check (BaggageStatus IN ('Checked-In', 'Loaded')),
62     BaggageFee decimal(10,2) default 0;
63

```

Additional Services Table

Handles passenger requests for supplemental services including extra baggage, better meals, and preferred seating. Each service is represented by a boolean variable, which includes a calculated total additional fare.

- Linked to the Ticket table via a foreign key.
- Enables dynamic pricing and charge monitoring based on service selections.
- Enables the efficient computation of overall revenue, including supplementary services.

Constraints and Best Practice

Each table utilises a mix of:

- Primary keys are used to uniquely identify records.
- Foreign keys ensure referential integrity among connected entities.
- Check the constraints to confirm that the values are genuine and business-acceptable.
- Default values are used when suitable (for example, dates and timings).
- Not using null constraints to ensure mandatory data fields.

This design technique prevents erroneous data, reduces redundancy, and increases database maintainability and reliability. Each table structure is consistent with the business logic of the airport's operational model.

```

223 |
224 |-- Inserting data into tables
225 -- Passenger Table
226 Insert into Passenger (PNR, FirstName, LastName, Email, DateOfBirth,MealPreference, EmergencyContact)
227 values ('PNR001', 'John', 'Smith', 'john.smith@email.com', '1980-05-12', 'Vegetarian', '1234567890'),
228 ('PNR002', 'Alice', 'Brown', 'alice.brown@email.com', '1992-07-19', 'Non-Vegetarian', '2345678901'),
229 ('PNR003', 'Robert', 'White', 'robert.white@email.com', '1978-10-21', 'Vegetarian', '3456789012'),
230 ('PNR004', 'Emily', 'Johnson', 'emily.johnson@email.com', '1985-12-03', 'Non-Vegetarian', '4567890123'),
231 ('PNR005', 'William', 'Davis', 'william.davis@email.com', '1995-09-17', 'Vegetarian', '5678901234'),
232 ('PNR006', 'Sophia', 'Miller', 'sophia.miller@email.com', '2000-03-25', 'Non-Vegetarian', '6789012345'),
233 ('PNR007', 'Michael', 'Wilson', 'michael.wilson@email.com', '1972-06-30', 'Vegetarian', '7890123456'),
234 ('PNR008', 'Olivia', 'Moore', 'olivia.moore@email.com', '1988-11-11', 'Non-Vegetarian', '8901234567'),
235 ('PNR009', 'David', 'Taylor', 'david.taylor@email.com', '1975-04-14', 'Vegetarian', '9012345678'),
236 ('PNR010', 'Jessica', 'Anderson', 'jessica.anderson@email.com', '1990-08-08', 'Non-Vegetarian', '1234567899')
237 |
238 -- Flight Table
239 Insert into Flight (FlightNumber, Origin, Destination, DepartureTime, ArrivalTime)
240 values
241 ('FL123', 'London', 'Paris', '2025-06-15 10:30:00', '2025-06-15 12:00:00'),
242 ('FL456', 'New York', 'Toronto', '2025-06-16 14:45:00', '2025-06-16 16:30:00'),
243 ('FL789', 'Dubai', 'Mumbai', '2025-06-17 18:00:00', '2025-06-17 21:00:00'),
244 ('FL1122', 'London', 'Delhi', '2025-06-18 10:30:00', '2025-06-18 23:30:00');
245 |
246 -- Resevation Table
247 Insert into Reservation(PNR, PassengerID,FlightId, BookingDate, BookingStatus)
248 values ('PNR001', '1', '1', '2025-06-01', 'Confirmed'),
249 ('PNR002', '2', '1', '2025-06-02', 'Pending'),
250 ('PNR003', '3', '2', '2025-06-03', 'Confirmed'),
251 ('PNR004', '4', '2', '2025-06-05', 'Cancelled'),
252 ('PNR005', '5', '3', '2025-06-06', 'Confirmed'),
253 ('PNR006', '6', '4', '2025-06-01', 'Pending'),
254 ('PNR007', '7', '4', '2025-06-01', 'Confirmed'),
255 ('PNR008', '8', '3', '2025-06-02', 'Pending'),
256 ('PNR009', '9', '2', '2025-06-03', 'Pending'),
257 ('PNR010', '10', '1', '2025-06-05', 'Cancelled');
258 |
259 -- Ticket Table
260 Insert into Ticket(ReservationId, IssueDate, SeatNumber, Class,Fare,EmployeeId)
261 values ('1', '2025-06-01', 'A1', 'Economy', '250', '1'),
262 ('3', '2025-06-03', 'B2', 'Business', '500', '2'),
263 ('5', '2025-06-06', 'C3', 'First-Class', '1000', '3'),
264 ('7', '2025-06-01', 'A3', 'Economy', '500', '3')
265 |
266 -- Empoyee Table
267 Insert into Employee(EmployeeName, EmployeeEmail, EmployeeRole)
268 values ('Sarah Lewis', 'sarah.lewis@email.com', 'Ticketing Staff'),
269 ('Tom Carter', 'tom.carter@email.com', 'Ticketing Supervisor'),
270 ('Rachel Adams', 'rachel.adams@email.com', 'Ticketing Staff')
271 |
272 |
273 -- Baggage Table
274 Insert into Baggage (TicketId, BaggageWeight, BaggageStatus,BaggageFee)
275 values ('1', '18', 'Checked-In', '0'),
276 ('2', '25', 'Checked-In', '500'),
277 ('3', '30', 'Loaded', '0'),
278 ('4', '20', 'Checked-In', '0')
279 |
280 |
281 -- Additional Services
282 Insert into AdditionalServices (TicketID,ExtraBaggage, UpgradedMeal, PreferredSeat,TotalAdditionalFare)
283 values ('1', '0', '1', '0', '20'),
284 ('2', '5', '0', '1', '530'),
285 ('3', '0', '1', '1', '50'),
286 ('4', '0', '0', '1', '30')
287 |

```

288 -- Checking data insertion

289 Select * from Passenger

290

83 %

Results Messages

	PassengerId	PNR	FirstName	LastName	Email	DateOfBirth	MealPreference	EmergencyContact
1	1	PNR001	John	Smith	john.smith@email.com	1980-05-12	Vegetarian	1234567890
2	2	PNR002	Alice	Brown	alice.brown@email.com	1992-07-19	Non-Vegetarian	2345678901
3	3	PNR003	Robert	White	robert.white@email.c...	1978-10-21	Vegetarian	3456789012
4	4	PNR004	Emily	Johnson	emily.johnson@email...	1985-12-03	Non-Vegetarian	4567890123
5	5	PNR005	William	Davis	william.davis@email.c...	1995-09-17	Vegetarian	5678901234
6	6	PNR006	Sophia	Miller	sophia.miller@email.c...	2000-03-25	Non-Vegetarian	6789012345
7	7	PNR007	Michael	Wilson	michael.wilson@email...	1972-06-30	Vegetarian	7890123456
8	8	PNR008	Olivia	Moore	olivia.moore@email.c...	1988-11-11	Non-Vegetarian	8901234567
9	9	PNR009	David	Taylor	david.taylor@email.com	1975-04-14	Vegetarian	9012345678
10	10	PNR010	Jessica	Anderson	jessica.anderson@e...	1990-08-08	Non-Vegetarian	1234567899

288 -- Checking data insertion

289 Select * from Flight

290

33 %

Results Messages

	FlightId	FlightNumber	Origin	Destination	DepartureTime	ArrivalTime
1	1	FL123	London	Paris	2025-06-15 10:30:00.000	2025-06-15 12:00:00.000
2	2	FL456	New York	Toronto	2025-06-16 14:45:00.000	2025-06-16 16:30:00.000
3	3	FL789	Dubai	Mumbai	2025-06-17 18:00:00.000	2025-06-17 21:00:00.000
4	4	FL1122	London	Delhi	2025-06-18 10:30:00.000	2025-06-18 23:30:00.000

287

288 -- Checking data insertion

289 Select * from Baggage

290

83 %

Results Messages

	BaggageId	TicketId	BaggageWeight	BaggageStatus	BaggageFee
1	1	1	18.00	Checked-In	0.00
2	2	2	25.00	Checked-In	500.00
3	3	3	30.00	Loaded	0.00
4	4	4	20.00	Checked-In	0.00

288 -- Checking data insertion

289 Select * from Reservation

290

83 %

Results Messages

	ReservationId	PNR	PassengerId	FlightId	BookingDate	BookingStatus
1	1	PNR001	1	1	2025-06-01	Confirmed
2	2	PNR002	2	1	2025-06-02	Pending
3	3	PNR003	3	2	2025-06-03	Confirmed
4	4	PNR004	4	2	2025-06-05	Cancelled
5	5	PNR005	5	3	2025-06-06	Confirmed
6	6	PNR006	6	4	2025-06-01	Pending
7	7	PNR007	7	4	2025-06-01	Confirmed
8	8	PNR008	8	3	2025-06-02	Pending
9	9	PNR009	9	2	2025-06-03	Pending
10	10	PNR010	10	1	2025-06-05	Cancelled

288 -- Checking data insertion

289 Select * from Ticket

290

83 %

Results Messages

	TicketId	ReservationId	IssueDate	IssueTime	SeatNumber	Class	Fare	EmployeeId	SeatStatus
1	1	1	2025-06-01	14:45:16.6866667	A1	Economy	250.00	1	Reserved
2	2	3	2025-06-03	14:45:16.6866667	B2	Business	500.00	2	Reserved
3	3	5	2025-06-06	14:45:16.6866667	C3	First-Class	1000.00	3	Reserved
4	4	7	2025-06-01	14:45:16.6866667	A3	Economy	500.00	3	Reserved

288 -- Checking data insertion

289 Select * from Employee

290

83 %

Results Messages

	EmployeeId	EmployeeName	EmployeeEmail	EmployeeRole
1	1	Sarah Lewis	sarah.lewis@email.com	Ticketing Staff
2	2	Tom Carter	tom.carter@email.com	Ticketing Supervisor
3	3	Rachel Adams	rachel.adams@email.com	Ticketing Staff
4	4	George Martin	george.martin@email.com	Ticketing Staff

288

-- Checking data insertion

289

Select * from AdditionalServices

290

83 %

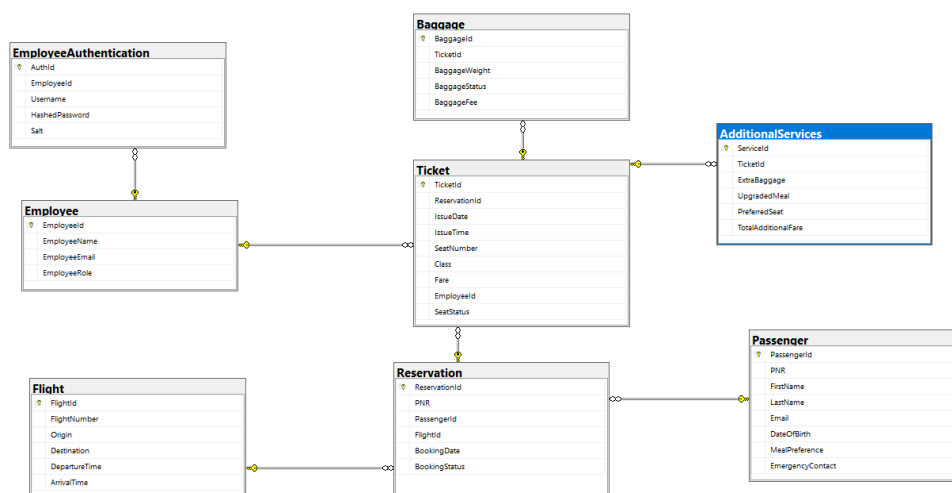
Results

Messages

	ServiceId	TicketId	ExtraBaggage	UpgradedMeal	PreferredSeat	TotalAdditionalFare
1	1	1	0.00	1	0	20.00
2	2	2	5.00	0	1	530.00
3	3	3	0.00	1	1	50.00
4	4	4	0.00	0	1	30.00

Entity-Relationship Diagram:

The airport ticketing system's Entity-Relationship (ER) Diagram is intended to reflect the actual flow of operations involved in booking, managing, and issuing airline tickets. The model focusses on the clear relationships between key system components such as passengers, employees, flights, reservations, tickets, baggage, and additional services.



Part 2: Solution to Questions (Q2–Q8)

Q2: Add the constraint to check that the reservation date is not in the past.

Solution: A check constraint was added to the Reservation table to ensure that reservations are not issued for expired dates. This constraint compares the reservation date to the system date using a function that evaluates the current date and forbids any entry with a reservation date less than today's date. This ensures that the data is logically consistent and prevents user error or data tampering, which could result in erroneous bookings. This validation is critical for real-time booking systems and adheres to the business requirement that only future reservations are valid.

Implementing this rule at the database level eliminates the possibility of circumventing it via external interfaces or manual queries.

```

297 |
298 | -- ques 2: Add the constraint to check that the reservation date is not in the past.
299 |
300 |
301 |
302 |
303 |
304 |
305 |
306 |
307 |
308 |
309 |
310 |
311 |
312 |
313 |
314 |
315 |
316 |
317 |
318 |
319 |
320 |
321 |
322 |
323 |
324 |
325 |
326 |
327 |
328 |
329 |
330 |
331 |
332 |
333 |
334 |
335 |
336 |
337 |
338 |
339 |
340 |
341 |
342 |
343 |
344 |
345 |
346 |
347 |
348 |
349 |
350 |
351 |
352 |
353 |
354 |
355 |
356 |
357 |
358 |
359 |
360 |
361 |
362 |
363 |
364 |
365 |
366 |
367 |
368 |
369 |
370 |
371 |
372 |
373 |
374 |
375 |
376 |
377 |
378 |
379 |
380 |
381 |
382 |
383 |
384 |
385 |
386 |
387 |
388 |
389 |
390 |
391 |
392 |
393 |
394 |
395 |
396 |
397 |
398 |
399 |
400 |
401 |
402 |
403 |
404 |
405 |
406 |
407 |
408 |
409 |
410 |
411 |
412 |
413 |
414 |
415 |
416 |
417 |
418 |
419 |
420 |
421 |
422 |
423 |
424 |
425 |
426 |
427 |
428 |
429 |
430 |
431 |
432 |
433 |
434 |
435 |
436 |
437 |
438 |
439 |
440 |
441 |
442 |
443 |
444 |
445 |
446 |
447 |
448 |
449 |
450 |
451 |
452 |
453 |
454 |
455 |
456 |
457 |
458 |
459 |
460 |
461 |
462 |
463 |
464 |
465 |
466 |
467 |
468 |
469 |
470 |
471 |
472 |
473 |
474 |
475 |
476 |
477 |
478 |
479 |
480 |
481 |
482 |
483 |
484 |
485 |
486 |
487 |
488 |
489 |
490 |
491 |
492 |
493 |
494 |
495 |
496 |
497 |
498 |
499 |
500 |
501 |
502 |
503 |
504 |
505 |
506 |
507 |
508 |
509 |
510 |
511 |
512 |
513 |
514 |
515 |
516 |
517 |
518 |
519 |
520 |
521 |
522 |
523 |
524 |
525 |
526 |
527 |
528 |
529 |
530 |
531 |
532 |
533 |
534 |
535 |
536 |
537 |
538 |
539 |
540 |
541 |
542 |
543 |
544 |
545 |
546 |
547 |
548 |
549 |
550 |
551 |
552 |
553 |
554 |
555 |
556 |
557 |
558 |
559 |
560 |
561 |
562 |
563 |
564 |
565 |
566 |
567 |
568 |
569 |
570 |
571 |
572 |
573 |
574 |
575 |
576 |
577 |
578 |
579 |
580 |
581 |
582 |
583 |
584 |
585 |
586 |
587 |
588 |
589 |
590 |
591 |
592 |
593 |
594 |
595 |
596 |
597 |
598 |
599 |
600 |
601 |
602 |
603 |
604 |
605 |
606 |
607 |
608 |
609 |
610 |
611 |
612 |
613 |
614 |
615 |
616 |
617 |
618 |
619 |
620 |
621 |
622 |
623 |
624 |
625 |
626 |
627 |
628 |
629 |
630 |
631 |
632 |
633 |
634 |
635 |
636 |
637 |
638 |
639 |
640 |
641 |
642 |
643 |
644 |
645 |
646 |
647 |
648 |
649 |
650 |
651 |
652 |
653 |
654 |
655 |
656 |
657 |
658 |
659 |
660 |
661 |
662 |
663 |
664 |
665 |
666 |
667 |
668 |
669 |
670 |
671 |
672 |
673 |
674 |
675 |
676 |
677 |
678 |
679 |
680 |
681 |
682 |
683 |
684 |
685 |
686 |
687 |
688 |
689 |
690 |
691 |
692 |
693 |
694 |
695 |
696 |
697 |
698 |
699 |
700 |
701 |
702 |
703 |
704 |
705 |
706 |
707 |
708 |
709 |
710 |
711 |
712 |
713 |
714 |
715 |
716 |
717 |
718 |
719 |
720 |
721 |
722 |
723 |
724 |
725 |
726 |
727 |
728 |
729 |
730 |
731 |
732 |
733 |
734 |
735 |
736 |
737 |
738 |
739 |
740 |
741 |
742 |
743 |
744 |
745 |
746 |
747 |
748 |
749 |
750 |
751 |
752 |
753 |
754 |
755 |
756 |
757 |
758 |
759 |
760 |
761 |
762 |
763 |
764 |
765 |
766 |
767 |
768 |
769 |
770 |
771 |
772 |
773 |
774 |
775 |
776 |
777 |
778 |
779 |
780 |
781 |
782 |
783 |
784 |
785 |
786 |
787 |
788 |
789 |
790 |
791 |
792 |
793 |
794 |
795 |
796 |
797 |
798 |
799 |
800 |
801 |
802 |
803 |
804 |
805 |
806 |
807 |
808 |
809 |
810 |
811 |
812 |
813 |
814 |
815 |
816 |
817 |
818 |
819 |
820 |
821 |
822 |
823 |
824 |
825 |
826 |
827 |
828 |
829 |
830 |
831 |
832 |
833 |
834 |
835 |
836 |
837 |
838 |
839 |
840 |
841 |
842 |
843 |
844 |
845 |
846 |
847 |
848 |
849 |
850 |
851 |
852 |
853 |
854 |
855 |
856 |
857 |
858 |
859 |
860 |
861 |
862 |
863 |
864 |
865 |
866 |
867 |
868 |
869 |
870 |
871 |
872 |
873 |
874 |
875 |
876 |
877 |
878 |
879 |
880 |
881 |
882 |
883 |
884 |
885 |
886 |
887 |
888 |
889 |
890 |
891 |
892 |
893 |
894 |
895 |
896 |
897 |
898 |
899 |
900 |
901 |
902 |
903 |
904 |
905 |
906 |
907 |
908 |
909 |
910 |
911 |
912 |
913 |
914 |
915 |
916 |
917 |
918 |
919 |
920 |
921 |
922 |
923 |
924 |
925 |
926 |
927 |
928 |
929 |
930 |
931 |
932 |
933 |
934 |
935 |
936 |
937 |
938 |
939 |
940 |
941 |
942 |
943 |
944 |
945 |
946 |
947 |
948 |
949 |
950 |
951 |
952 |
953 |
954 |
955 |
956 |
957 |
958 |
959 |
960 |
961 |
962 |
963 |
964 |
965 |
966 |
967 |
968 |
969 |
970 |
971 |
972 |
973 |
974 |
975 |
976 |
977 |
978 |
979 |
980 |
981 |
982 |
983 |
984 |
985 |
986 |
987 |
988 |
989 |
990 |
991 |
992 |
993 |
994 |
995 |
996 |
997 |
998 |
999 |
1000 |

```

Q3: Identify Passengers with Pending Reservations and Passengers with age more than 40 years.

Solution: This query returns passengers that satisfy one of two specified conditions:

- Their reservation has the status "Pending" (not yet confirmed or finalised), or
- They are more than forty years old.

The implementation employs a JOIN between the Passenger and Reservation columns, with criteria on Status and Age. The use of logical operators guarantees correct filtering based on business principles.

This query improves customer service and analytics by identifying customers who require attention (e.g., follow-up calls for pending bookings) and providing specialised services for elder passengers (e.g., special help).

```

300 | -- ques 3: Identify Passengers with Pending Reservations and Passengers with age more than 40 years.
301 | Select p.PassengerId,p.FirstName,p.LastName, DATEDIFF(YEAR,p.DateOfBirth,GETDATE()) as age
302 | from Passenger p join Reservation r on p.PassengerId=r.PassengerId
303 | where DATEDIFF(YEAR,p.DateOfBirth,GETDATE())>40 and r.BookingStatus='Pending';

```

PassengerId	FirstName	LastName	age
9	David	Taylor	50

Q4: The ticketing system also requires stored procedures or user-defined functions to do the following things:

- Search the database of the ticketing system for matching character strings by last name of passenger. Results should be sorted with most recent issued ticket first.
- Return a full list of passengers and his/her specific meal requirement in business class who has a reservation today (i.e., the system date when the query is run).
- Insert a new employee into the database.
- Update the details for a passenger that has booked a flight before.

Solution: a) A stored procedure was developed to search for passengers by last name utilising pattern matching (LIKE) and arranging the results by the most recent ticket

issue date. It connects the Passenger and Ticket tables and sorts by IssueDate in descending order. This technology allows customer care agents to quickly retrieve a customer's most recent trip information, improving user experience and operational speed.

```

305 -- ques 4: a) Search the database of the ticketing system for matching character strings by last name of passenger.
306 -- Results should be sorted with most recent issued ticket first.
307 Create procedure SearchPassengerByLastName
308 @LastName nvarchar(50)
309 as
310 begin
311 Select p.PassengerId, p.Firstname, p.Lastname, t.TicketId, t.Issuedate
312 from Passenger p join Reservation r on p.PassengerId=r.PassengerId
313 join Ticket t on r.ReservationId=t.ReservationId
314 where p.Lastname like '%'+@LastName+'%'
315 order by t.Issuedate desc
316 end
317
318 exec SearchPassengerByLastName 'son'
319
33 %

```

	PassengerId	Firstname	Lastname	TicketId	Issuedate
1	7	Michael	Wilson	4	2025-06-01

b) A user-defined function returns passengers travelling in business class who have bookings on the current system date, along with their meal selections. This entails filtering by class type and comparing the reservation date to the system date. This function assists in handling in-flight service logistics, such as producing customised meals based on dietary requirements, and guarantees that business class passengers receive high-quality service.

```

320 -- b) Return a full list of passengers and his/her specific meal requirement in business class who has a reservation today
321 -- (i.e., the system date when the query is run)
322 Create function GetBusinessClassPassengersToday()
323 returns table
324 as
325 return
326 Select p.PassengerId, p.Firstname, p.Lastname, p.Mealpreference, r.Bookingdate, t.class
327 from Passenger p join Reservation r on p.PassengerId=r.PassengerId
328 join Ticket t on r.ReservationId=t.ReservationId
329 where t.class='Business' and r.Bookingdate=cast(GETDATE() AS DATE);
330
331 select* from GetBusinessClassPassengersToday()

```

	PassengerId	Firstname	Lastname	Mealpreference	Bookingdate	class
--	-------------	-----------	----------	----------------	-------------	-------

c) A stored procedure encapsulates the logic for adding a new employee to the database. It guarantees that all needed fields are checked and properly entered, reducing human error and enforcing business rules. This approach secures and streamlines HR operations by handling data validation and preventing unauthorised tampering via ad hoc queries.

```

333 -- c) Insert a new employee into the database.
334 Create procedure InsertNewEmployee
335     @EmployeeName nvarchar(100),
336     @EmployeeEmail nvarchar(100),
337     @EmployeeRole nvarchar(20)
338 as
339 begin
340     Insert into Employee(EmployeeName, EmployeeEmail, EmployeeRole)
341     values (@EmployeeName, @EmployeeEmail, @EmployeeRole)
342 end;
343
344 EXEC InsertNewEmployee 'George Martin', 'george.martin@email.com', 'Ticketing Staff';
345 Select* from Employee

```

83 %

Results Messages

	EmployeeId	EmployeeName	EmployeeEmail	EmployeeRole
1	1	Sarah Lewis	sarah.lewis@email.com	Ticketing Staff
2	2	Tom Carter	tom.carter@email.com	Ticketing Supervisor
3	3	Rachel Adams	rachel.adams@email.com	Ticketing Staff
4	4	George Martin	george.martin@email.com	Ticketing Staff

d) Update Existing Passenger Information: A saved mechanism is provided for updating existing passenger records. It allows passengers to alter personal information such as email, meal preference, or emergency contact if they have previously booked a reservation. This ensures that updates are only sent to active clients and improves data dependability by verifying reservation history prior to conducting updates.

```

347 -- d) Update the details for a passenger that has booked a flight before.
348 Create procedure UpdatePassengerDetails
349     @PassengerId int,
350     @FirstName nvarchar(50),
351     @LastName nvarchar(50),
352     @Email nvarchar(50),
353     @MealPreference nvarchar(15)
354 as
355 begin
356     if exists (Select 1 from Reservation where PassengerId = @PassengerId)
357     begin
358         update Passenger
359         set FirstName = @FirstName,
360             LastName = @LastName,
361             Email = @Email,
362             MealPreference = @MealPreference
363         where PassengerId = @PassengerId;
364     end
365     else
366     begin
367         print 'Passenger has no reservations and cannot be updated.';
368     end
369 end;
370

```



```

371 EXEC UpdatePassengerDetails
372 @PassengerId = 11,
373 @FirstName = 'Robert',
374 @LastName = 'White',
375 @Email = 'new.robert@email.com',
376 @MealPreference = 'Vegetarian';

```

5 %

Messages

Passenger has no reservations and cannot be updated.

Q5: The ticketing system wants to be able to view all e-boarding numbers Issued by a Specific Employee showing the overall revenue generated by that employee on a particular flight. It should include details of the fare, additional baggage fees, upgraded meal or preferred seat. You should create a view containing all the required information.

Solution: A VIEW is developed to aggregate and display:

- Boarding pass numbers issued by a specific employee.
- Total revenue generated per flight by that employee.
- Fare information, luggage costs, and prices for extra services (meal upgrades, seat selection).
- This view combines the Ticket, Employee, Flight, Baggage, and AdditionalServices columns and groups data to calculate total contributions per employee each flight.

The goal is to track employee performance and determine operational income per employee, which is critical for management and financial reporting.

```

378 -- ques 5:
379 /* The ticketing system wants to be able to view all e-boarding numbers Issued by a Specific
380 Employee showing the overall revenue generated by that employee on a particular
381 flight. It should include details of the fare, additional baggage fees, upgraded meal or
382 preferred seat. You should create a view containing all the required information */
383 --Create view EmployeeTicketRevenue as
384 Create View EmployeeTicketRevenue as
385 Select e.EmployeeId,e.EmployeeName,t.TicketId as EboardingNumber, f.flightId,f.FlightNumber,
386 sum(t.Fare) as TotalTicketFare ,sum( ISNULL(b.BaggageFee,0)) as BaggageFee,sum( ISNULL(a.TotalAdditionalFare,0)) as TotalAdditionalFare,
387 sum((t.Fare + ISNULL(a.TotalAdditionalFare, 0))) AS TotalRevenuePerTicket
388 from Ticket t join Reservation r on t.ReservationId=r.ReservationId join Flight f on r.FlightId = f.FlightId
389 join Employee e on t.EmployeeId = e.EmployeeId
390 left join Baggage b on t.TicketId = b.TicketId
391 left join AdditionalServices a on t.TicketId = a.TicketId
392 group by E.EmployeeId, E.EmployeeName, F.FlightId, F.FlightNumber,t.ticketId
393
394 Select * from EmployeeTicketRevenue
395 where EmployeeId = 3 and FlightNumber = 'FL789';
396

```

5 %

Results Messages

EmployeeId	EmployeeName	EboardingNumber	flightId	FlightNumber	TotalTicketFare	BaggageFee	TotalAdditionalFare	TotalRevenuePerTicket
3	Rachel Adams	3	3	FL789	1000.00	0.00	50.00	1050.00

Q6: Create a trigger so that the current seat allotment of a passenger automatically updates to reserved when the ticket is issued.

Solution: A trigger is added to the Ticket database to automatically update the SeatAllotment field in the reservation when a new ticket is issued. When a seat is

confirmed through ticket generation, its status switches from "available" or "pending" to "reserved". This ensures that seat availability is accurate in real time, preventing multiple bookings and assisting scheduling departments in managing seat inventory effectively. Triggers ensure that this key business rule is enforced automatically and without user participation.

```

397 -- Ques 6: Create a trigger so that the current seat allotment of a passenger automatically updates to reserved when the ticket is issued.
398 Alter table Ticket add SeatStatus nvarchar(10)
399 default 'Available' check (SeatStatus in ('Available', 'Reserved'));
400 update Ticket set SeatStatus='Reserved'
401 select * from Ticket

```

75 %

Results Messages

	TicketId	ReservationId	IssueDate	IssueTime	SeatNumber	Class	Fare	EmployeeId	SeatStatus
1	1	1	2025-06-01	14:45:16.6866667	A1	Economy	250.00	1	Reserved
2	2	3	2025-06-03	14:45:16.6866667	B2	Business	500.00	2	Reserved
3	3	5	2025-06-06	14:45:16.6866667	C3	First-Class	1000.00	3	Reserved
4	4	7	2025-06-01	14:45:16.6866667	A3	Economy	500.00	3	Reserved

```

402
403 Create trigger UpdateSeatStatusOnIssue
404 on Ticket
405 after insert
406 as
407 begin
408     update T
409     set SeatStatus = 'Reserved'
410     from Ticket T
411     inner join inserted i on T.TicketId = i.TicketId
412     inner join Reservation R on i.ReservationId = R.ReservationId
413     where R.BookingStatus = 'Confirmed';
414 end;
415

```

Q7: You should provide a function or view that allows the ticketing system to identify the total number of baggage items (which are checked in) made on a specified date for a specific flight.

Solution: A user-defined function is used to compute the total number of checked-in luggage for a specific flight on a given date. It receives the aircraft ID and date as inputs and aggregates the baggage data accordingly.

This assists ground crews and flight operations in preparing for cargo loading, weight distribution, and logistics. It also assures adherence to airline safety and efficiency rules.

```

416
417 -- Ques 7: You should provide a function or view which allows the ticketing system to
418 -- identify the total number of baggage's (which are checkedin) made on a specified date for a specific flight.
419 Create view BaggageSummary as
420 Select r.FlightId,t.IssueDate, count(b.BaggageId) as TotalBaggageCheckedIn
421 from Baggage b inner join Ticket t on b.TicketId=t.TicketId inner join Reservation r on t.ReservationId=r.ReservationId
422 where b.BaggageStatus='Checked-In' group by r.FlightId,t.IssueDate
423
424 SELECT * FROM BaggageSummary
425 WHERE FlightId = 2 AND IssueDate = '2025-06-03';
426
75 %
Results Messages
FlightId IssueDate TotalBaggageCheckedIn
1 2 2025-06-03 1

```

Q8: If there are any other database objects such as views, stored procedures, user-defined functions, or triggers which you think would be relevant to the airport scenario given the brief above, you will obtain higher marks for providing these along with an explanation of their functionality.

```

427 -- Ques 8: If there are any other database objects such as views, stored procedures, user-defined
428 /*functions, or triggers which you think would be relevant to the airport scenario given
429 the brief above, you will obtain higher marks for providing these along with an
430 explanation of their functionality. */

```

- To automatically reject baggage entries that exceed the 20kg free allowance without appropriate baggage fee in Baggage table.
 - This trigger is intended to impose a tight business norm for luggage handling. It specifically requires that any baggage that exceeds the free weight allowance of 20kg include an appropriate baggage tax. If this rule is broken, the insert action is reversed, and a user-friendly error message is generated.
 - Revenue Loss Prevention: The method prevents revenue leakage by rejecting unpaid overweight baggage.
 - Operational Efficiency: Ground staff may be confident that only valid baggage entries are in the system, decreasing the need for human checks.
 - Improved Data Integrity: Maintains consistency in the relationship between baggage weight and costs.

```

432 -- query 1:( To automatically reject baggage entries that exceed the 20kg free allowance without appropriate baggage fee in Baggage table.)
433 Create trigger PreventOverweightBaggageWithoutFee
434 on Baggage
435 instead of insert
436 as
437 begin
438 If exists (
439 select 1
440 from inserted
441 where BaggageWeight > 20 AND BaggageFee < (BaggageWeight - 20) * 100
442 )
443 begin
444 raiserror('Overweight baggage must include appropriate additional fee.', 16, 1)
445 rollback transaction
446 return
447 end
448
449 -- If all good, insert the baggage
450 Insert into Baggage (TicketId, BaggageWeight, BaggageStatus, BaggageFee)
451 Select TicketId, BaggageWeight, BaggageStatus, BaggageFee
452 from inserted
453 end;
454

```

- Find passengers who spent more than £1000 (fare + services) in total.
 - This query uses important transactional tables (Passenger, Reservation, Ticket, and AdditionalServices) to compute each passenger's total cost. Passengers having a total of more than £1000 are returned with their information and estimated total.
 - Customer Profiling: Identifies VIP or high-value passengers who may be eligible for loyalty programs or upgrades.
 - Marketing insight enables targeted promotions, discounts, and personalised offers.
 - Revenue Monitoring: Gives finance and business teams insight into which passengers generate the most revenue.
 - Customer service and marketing teams can utilise this knowledge to automatically enrol high-spending travellers in premium programs, as well as create loyalty bonuses and outreach initiatives.
 - Why This Matters: Modern airlines rely largely on data to customise passenger experiences. A high-spending passenger could be a regular flyer, a business traveler, or someone who consistently selects expensive extras. Understanding these trends enables the organisation to strengthen ties and ensure long-term commitment.

```

455 -- query 2(Find passengers who spent more than £1000 (fare + services) in total.)
456 Select
457     P.PNR,
458     P.FirstName,
459     P.LastName,
460     SUM(T.Fare + ISNULL(A.TotalAdditionalFare, 0)) as TotalSpent
461 from Passenger P
462 join Reservation R on P.PassengerId = R.PassengerId
463 join Ticket T on R.ReservationId = T.ReservationId
464 left join AdditionalServices A on T.TicketId = A.TicketId
465 group by P.PNR, P.FirstName, P.LastName
466 having SUM(T.Fare + ISNULL(A.TotalAdditionalFare, 0)) > 1000;
467

```

PNR	FirstName	LastName	TotalSpent
PNR003	Robert	White	1030.00
PNR005	William	Davis	1050.00

Additional Recommendations

Secure Authentication and Role-Based Access Management

To improve system security and assure acceptable user access to the airport ticketing platform, a number of robust techniques were built under the EmployeeAuthentication table using stored procedures. These mechanisms enable secure credential management, role-based control, and operational accountability.

The authentication framework provides a method for registering new employee credentials, which ensures that usernames are unique and that only existing employees can be linked to login records. The employee's password is hashed with

the SHA2-512 method and a randomly generated salt value before being saved in the database. This strategy reduces the danger of password leakage while also adhering to password security best practices.

A second technique is used for login validation, in which the system securely gets the stored salt for a specific username, recomputes the hash with the user's password, and compares it to the saved hash. Following successful authentication, essential employee information is obtained, granting access to the ticketing system. This ensures that only approved personnel may interface with important modules like reservations, ticket issuing, and passenger management.

To implement role-based security, an extra method requires only employees with the "Ticketing Supervisor" role to reset the passwords of other employees. Before upgrading login credentials, the system validates the supervisor's credentials and role, ensuring clear administrative authority boundaries and reducing unauthorised intrusion.

This design not only promotes system integrity, but it also allows for future scalability when adding more complex security layers like multi-factor authentication, audit logging, or temporary access tokens. Overall, the combination of procedural encapsulation and hashed credential management improves the platform's security posture while also enabling the operational workflows of both ticketing workers and supervisors.

```
83 | -- Set employee password
84 | Create procedure AddAuthenticationForEmployee
85 | @EmployeeId int,
86 | @Username nvarchar(50),
87 | @Password nvarchar(50)
88 | as
89 | begin
90 |     set nocount on;
91 |     -- Check if employee exists
92 |     If not exists (select 1 from Employee where EmployeeId = @EmployeeId)
93 |     begin
94 |         raiserror('Employee does not exist.', 16, 1);
95 |         return;
96 |     end
97 |     -- Check if username already used
98 |     If exists (select 1 from EmployeeAuthentication where Username = @Username)
99 |     begin
100 |         raiserror('Username already taken.', 16, 1);
101 |         return;
102 |     end
```

```

103 declare @Salt uniqueidentifier = NEWID();
104 declare @SaltStr nvarchar(36) = CAST(@Salt as nvarchar(36));
105 declare @HashedPassword binary(64);
106
107 Set @HashedPassword = HASHBYTES('SHA2_512', @Password + @SaltStr);
108
109 -- Insert into Authentication table
110 Insert into EmployeeAuthentication (EmployeeId, Username, HashedPassword, Salt)
111 values (@EmployeeId, @Username, @HashedPassword, @SaltStr);
112 end
113
114 Exec AddAuthenticationForEmployee
115     @EmployeeId = 1,
116     @Username = 'SL201',
117     @Password = 'LewisP@ssword!';

```

```

119 -- Login
120 Create procedure AuthenticateEmployee
121     @Username nvarchar(50),
122     @Password nvarchar(100)
123 as
124 begin
125     Set nocount on;
126     declare @StoredSalt nvarchar(40);
127     declare @StoredHash binary (64);
128     declare @ComputedHash binary(64);
129     declare @EmployeeId int;
130
131     -- Get salt and hash for the provided username
132     Select top 1
133         @StoredSalt = Salt,
134         @StoredHash = HashedPassword,
135         @EmployeeId = EmployeeId
136     from EmployeeAuthentication
137     where Username = @Username;
138

```

```

139 -- If no matching user, return error
140 If @StoredSalt is null
141 begin
142     print 'Authentication failed: User not found.';
143     return;
144 end
145 -- Recompute hash using input password + stored salt
146 set @ComputedHash = HASHBYTES('SHA2_512', @Password + @StoredSalt);
147 -- Compare the hashes
148 If @ComputedHash = @StoredHash
149 begin
150     -- Authentication successful — fetch and return details
151     Select
152         e.EmployeeId,
153         e.EmployeeName,
154         e.EmployeeEmail,
155         e.EmployeeRole
156     from Employee e
157     where e.EmployeeId = @EmployeeId;
158 end
159 else
160 begin
161     -- Authentication failed — wrong password
162     print 'Authentication failed: Incorrect password. Ticketing Supervisor may intervene.';
163 end
164 end

166 Exec AuthenticateEmployee
167     @username='SL201',
168     @Password='NewSecurePass123'
169

-- checking role and granting permission
-- Only Ticketing Supervisor can reset passwords
174 Create procedure ResetEmployeePassword
175     @SupervisorId int,
176     @TargetEmployeeId int,
177     @NewPassword nvarchar(100)
178 as
179 begin
180     declare @SupervisorRole nvarchar(20);
181     declare @HashedPassword binary(64);
182     declare @Salt uniqueidentifier = NEWID();
183     declare @SaltStr nvarchar(36) = CAST(@Salt as nvarchar(36));
184

185     -- Check if the supervisor exists and has the right role
186     Select @SupervisorRole = EmployeeRole
187     from Employee
188     where EmployeeId = @SupervisorId;
189
190     If @SupervisorRole is null
191     begin
192         throw 50001, 'Supervisor not found.', 1;
193         return;
194     end
195
196     If @SupervisorRole != 'Ticketing Supervisor'
197     begin
198         throw 50002, 'Permission denied. Only Ticketing Supervisors can reset passwords.', 1;
199         return;
200     end
201

```



```

202 -- Hash the new password
203 Set @HashedPassword = HASHBYTES('SHA2_512', @NewPassword + @SaltStr);
204
205 -- Update the password for the target employee
206 Update EmployeeAuthentication
207 Set
208     HashedPassword = @HashedPassword,
209     Salt = @SaltStr
210 where EmployeeId = @TargetEmployeeId;
211
212 select 'Password reset successfully by Supervisor.' as message;
213 end;
214
215
216 -- Supervisor (EmployeeId 2) resets password for EmployeeId 1
217 Exec ResetEmployeePassword @SupervisorId = 2, @TargetEmployeeId = 1, @NewPassword = 'NewSecurePass123';
218

```

```

166 Exec AuthenticateEmployee
167     @username='SL201',
168     @Password='NewSecurePass123'
169
170
171

```

3 %

EmployeeId	EmployeeName	EmployeeEmail	EmployeeRole
1	Sarah Lewis	sarah.lewis@email.com	Ticketing Staff

Data Integrity and Concurrency

Managing concurrent processes and guaranteeing data integrity are critical in a high-volume transactional system such as an airline reservation platform. The following tactics were used to emphasise these principles:

- Primary and foreign key restrictions are used to specify all major relationships, preventing orphan records and ensuring relational accuracy.
- Check Constraints: Field constraints such as reservation status, luggage weight limits, and class types ensure that only valid entries are accepted, which reduces data entering errors.
- Default constraints are used for columns such as ticket issuing date and time to ensure consistent and efficient data entry.
- Transactions: Operations such as luggage entry and passenger updates are encased in BEGIN TRANSACTION blocks to ensure atomicity and avoid partial modifications.
- Triggers are used to enforce business rules such as fee validation for overweight baggage, which eliminates the need for frequent manual checks.
- Concurrency Control: Isolation levels such as READ COMMITTED are enabled by default to avoid dirty reads and ensure consistency during concurrent access.

These strategies work together to eliminate anomalies and safeguard against data corruption in multiuser scenarios.

Database Security

Given that the system handles sensitive information regarding passengers, staff, and revenue, numerous steps were taken to assure database security:

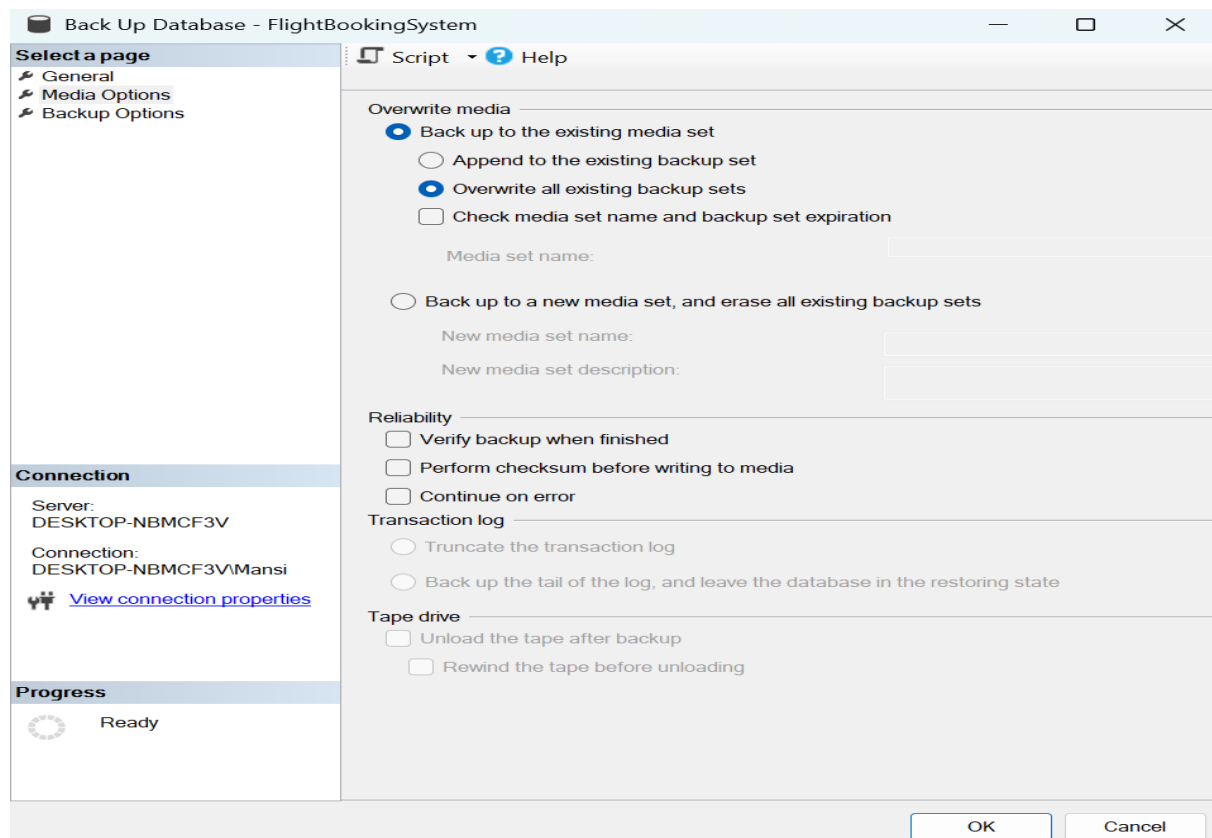
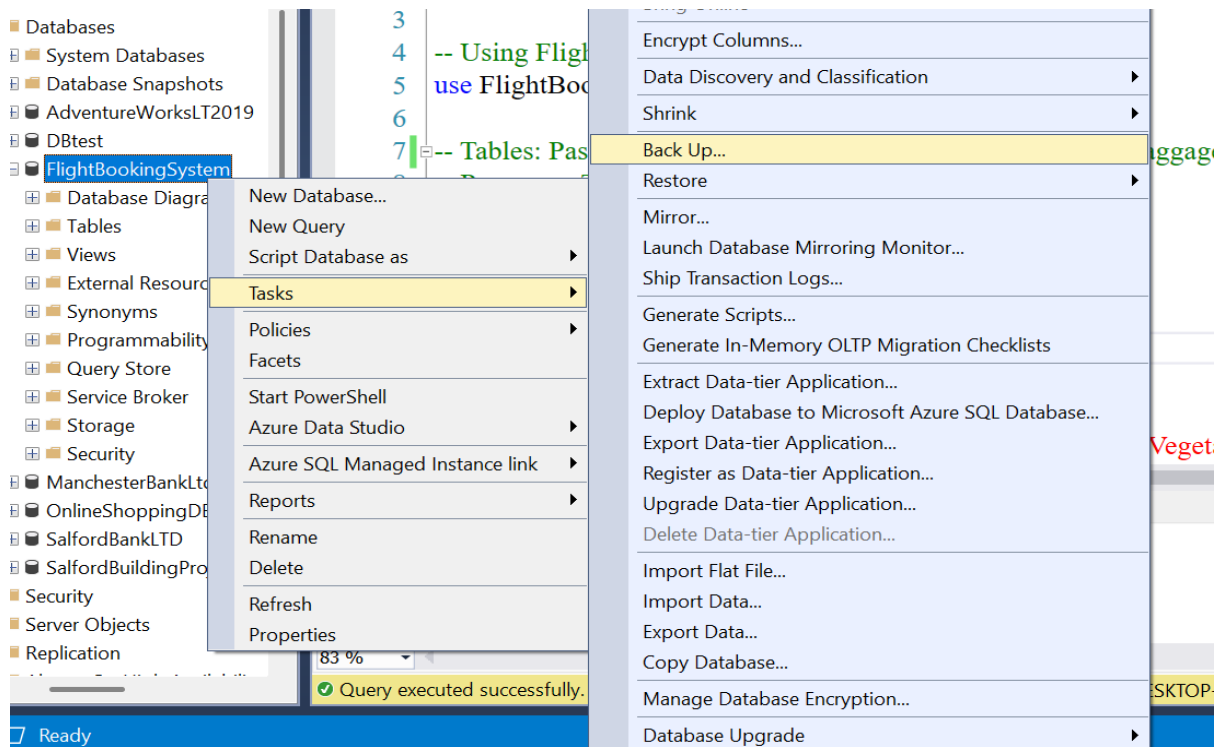
- The EmployeeAuthentication Table stores secure login credentials for system users, ensuring that only authorised personnel can access sensitive processes.
- Role-based Access Control: Although not enforced programmatically in this implementation, the schema is designed to provide access limitations based on job function, such as separating ticket agents from administrator users.
- Sensitive fields are withheld or masked in views to protect data while still allowing for functionality. For example, revenue views may show aggregated figures but not individual fares.
- Stored Procedures: By routing all data alterations through stored procedures, the system lessens the vulnerability of underlying tables and the possibility of SQL injection attacks.
- Validation Triggers: These detect and prevent anomalies at the data layer, lowering the risk of invalid inputs or malicious alterations.

Backup and Recovery

To avoid losing essential data due to hardware failure, human mistake, or cyber-attacks, the following measures are recommended for a production implementation:

- Full database backups should be scheduled every day during off-peak hours.
- Differential backups are used to record changes made since the last full backup, allowing for speedier recovery and reduced storage demand.
- Transaction Log Backups: Allow for point-in-time recovery, which is very crucial in a booking system where every second might impact availability.
- Backups should be copied off-site or saved on secure cloud infrastructure to provide redundancy.
- Automated Scripts: SQL Server Agent Jobs can be used to perform backup and integrity tests, such as consistency checks and index maintenance.

Together, these practices ensure that data loss is minimized and the database remains resilient against both system-level failures and operational oversights.



Back Up Database - FlightBookingSystem

Select a page

General

Media Options

Backup Options

Script

Help

Backup set

Name:FlightBookingSystem-Full Database Back

Description:

Backup set will expire:

After:

0

days

On:

21/04/2025

Compression

Set backup compression:Compress backup

Encryption

Encrypt backup

Algorithm:AES 128

Certificate or Asymmetric key:

Encryption is available only when Back up to a new media set is selected in Media Options.

Connection

Server:DESKTOP-NBMCF3V

Connection:DESKTOP-NBMCF3V\Mansi

View connection properties

Progress

Ready

OK

Cancel

Back Up Database - FlightBookingSystem

Select a page

General

Media Options

Backup Options

Script

Help

Source

Database:FlightBookingSystem

Recovery model:FULL

Backup type:Full

Copy-only backup

Backup component:

Database

Files and filegroups:

Destination

Back up to:Disk

C:\mansi\University\Trimester_2\ADB\Backup\FlightBookingSystem.bak

Add...

Remove

Contents

Connection

Server:DESKTOP-NBMCF3V

Connection:DESKTOP-NBMCF3V\Mansi

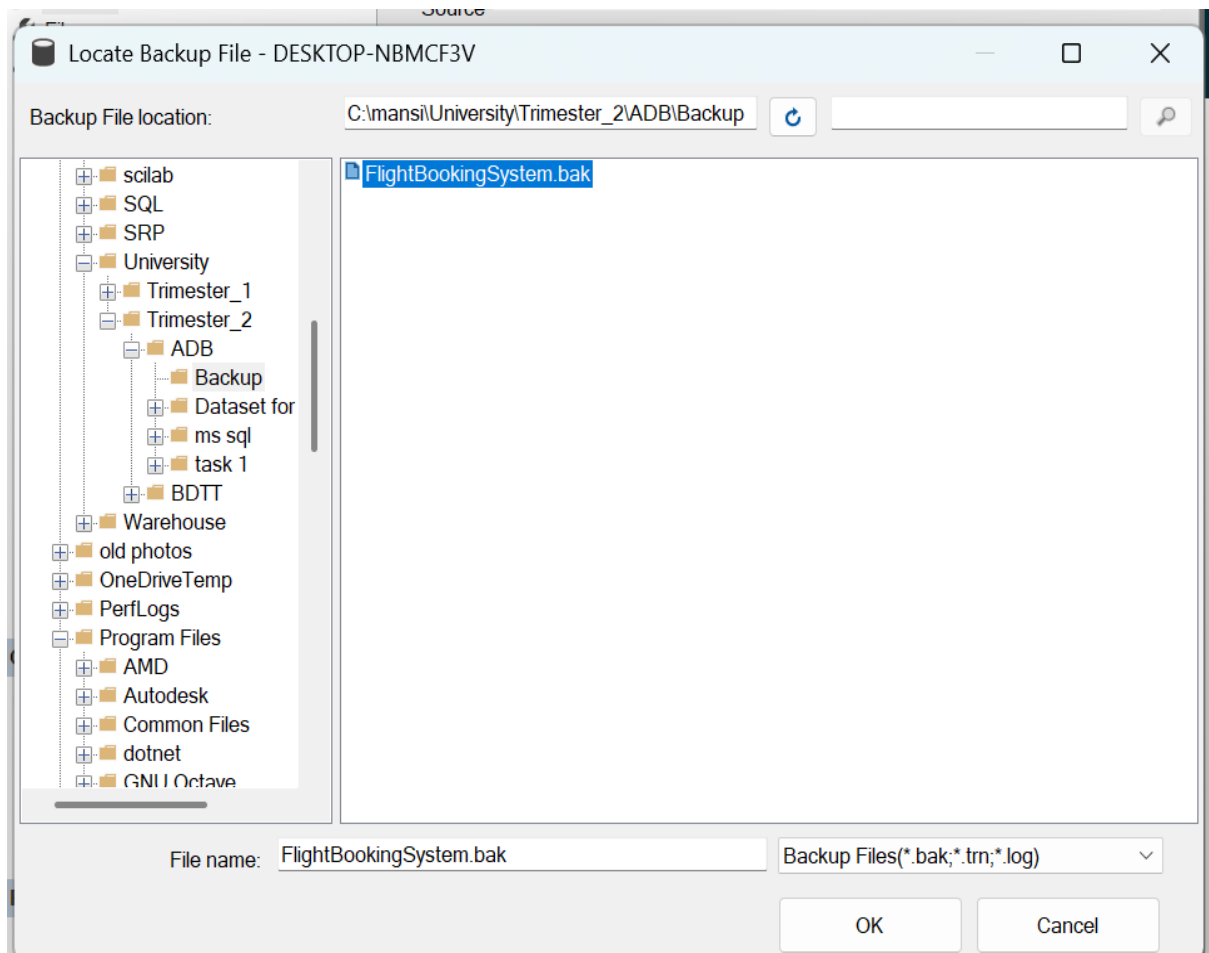
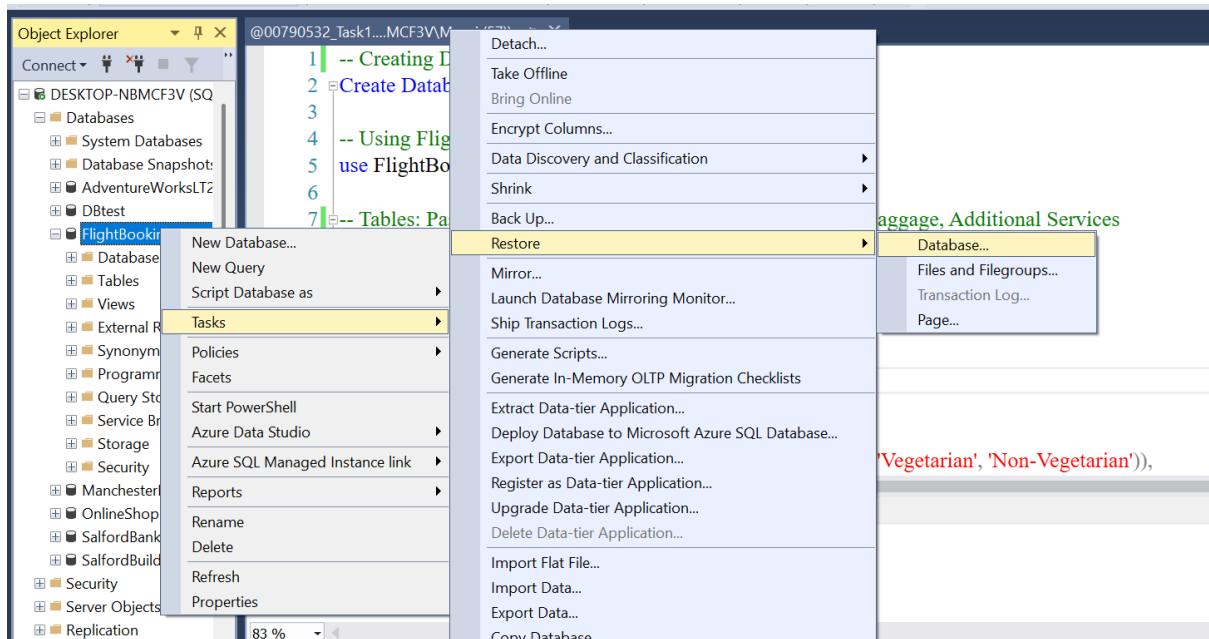
View connection properties

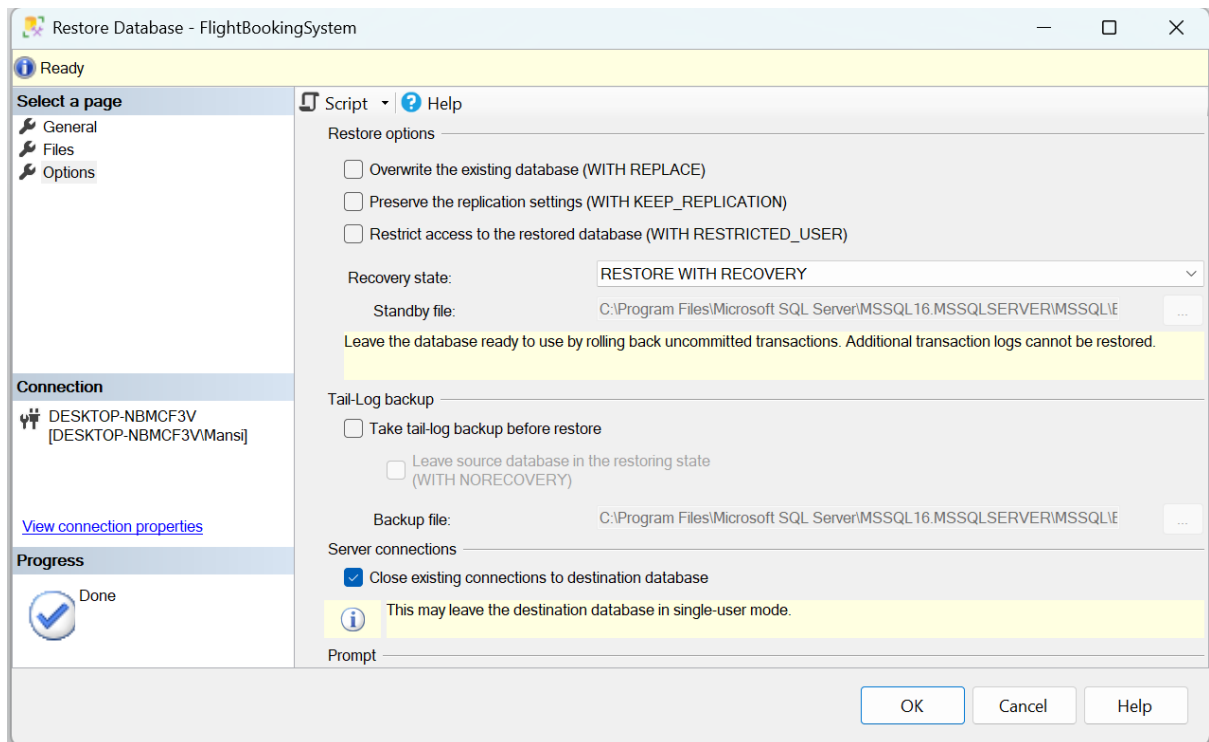
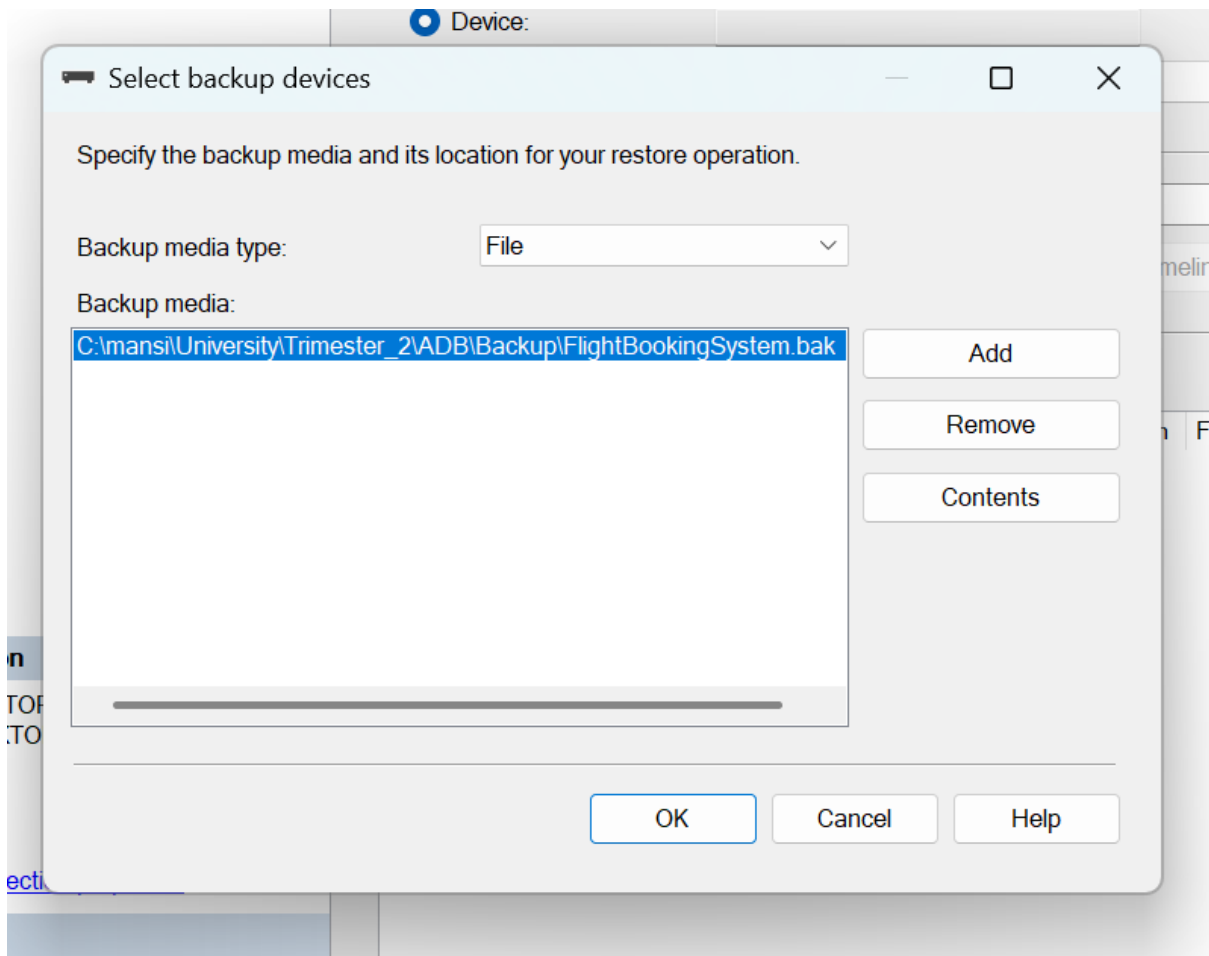
Progress

Ready

OK

Cancel





Conclusions

The flight booking and ticketing database created for this project is highly functional, reliable, and maintainable. It includes a wide range of fundamental airline ticketing functions, including as bookings, seat assignments, supplementary services, luggage tracking, and employee control, all within a relational database system that is normalised to 3NF.

The key highlights of this implementation are:

- A comprehensive schema with logical linkages among passengers, planes, tickets, staff, and services.
- Triggers, views, stored procedures, and user-defined functions that enforce business logic and optimise data access are all examples of advanced T-SQL techniques.
- Real-world problem-solving tasks include detecting high-value customers, validating baggage fees, and entering data using constraints.
- A scalable design that can be expanded to include features such as loyalty programs, mobile check-in, and automatic alarms.
- Concentrate on best practices including constraint enforcement, normalised design, access control, and data backup strategy.

This system provides a solid foundation for installing a real-world airline reservation system and exhibits a thorough understanding of relational databases, practical SQL usage, and enterprise-grade design concepts. It provides both operating efficiency and scalability for future growth, meeting all of the requirements of a contemporary airline infrastructure.